PROGRAMMING METHODS - LABORATORY 4

Program 01

Write and test a program that demonstrates how to implement a stack:

- in a one-dimensional dynamic array;
- using a singly linked list;
- using the stack adapter - use std::stack from the C++ Standard Template Library.

Each implementation should support the following operations: checking whether the stack is empty, returning the number of elements stored on the stack, returning the value of the top element, placing a new element on the top of the stack, and removing the existing top element.

Input

A test file with any name (standard input) has the following format: the first line contains an integer n denoting the number of elements. The next line contains n integers, which are the consecutive elements.

Output

A demonstration of how each operation works.

Program 02

Write a memory- and computation-efficient program performing the following operations:

- conversion of arithmetic expressions and assignment statements from traditional infix notation to Reverse Polish Notation (ONP/RPN);
- conversion of arithmetic expressions and assignment statements from ONP/RPN to infix notation with the minimum number of parentheses used.

An assignment statement has the form: operand = arithmetic expression. Arithmetic expressions may contain only parentheses (and) in infix notation, and operands that are lowercase letters of the English alphabet.

Operator	Priority	Associativity	Operator type
=	0	right	assignment
< >	1	left	relational
+ -	2	left	additive
* / %	3	left	multiplicative
^	4	right	exponentiation
~	5	right	unary

Input for Program 02

Data are stored in a text file containing consecutive lines. The first line contains an integer from 1 to 2^15, denoting the number of lines containing arithmetic expressions.

Each line contains at least 6 and at most 256 characters and may have one of two forms:

- INF: an arithmetic expression or assignment statement written in infix notation;
- ONP: an arithmetic expression or assignment statement written in ONP/RPN notation.

The expressions may contain any characters. The program first removes characters that do not occur in valid expressions, including spaces, and then checks expression correctness. You may assume that after removing invalid symbols, INF expressions are correct if they are correct in C++. ONP expressions are correct if they are executable.

Output for Program 02

An input expression preceded by "INF:" must be preceded by "ONP:" in the output. Similarly, an input expression preceded by "ONP:" must be preceded by "INF:" in the output. If an expression is invalid, output the word "error" instead of the converted expression.

When converting to infix notation, the output expression must contain the minimum number of parentheses that guarantees the same order of operations as in the input expression. For example, ONP: abc** is converted to INF: a*(b*c), not INF: a*b*c, because the parentheses force the multiplication order present in the ONP input.

Example: for INF input such as INF:(a,+ b)/..[c3, the program keeps only (a+b)/c, discards the other characters including spaces, checks correctness, and outputs ONP: ab+c/. For ONP input such as ONP: (a,b,.).c;-,* the program keeps only abc-*, checks correctness, and outputs INF: a*(b-c).

Examples

Input

```
18
INF: a)+(b
ONP: ab+a~a-+
INF: a+b+(~a-a)
INF: x=~~a+b*c
INF: t = ~ a < x < ~b
INF: ~a-~~b<c+d&!p|!!q
INF: a^b*c-d<xp|q+x
INF: x=~a*b/c-d+e%~f
ONP: xabcdefg++++++=
ONP: ab+c+d+e+f+g+
ONP: abc++def++g++
ONP: abc++def++g+++
INF: x=a=b=c
ONP: xabc===
INF: x=a^b^c
INF: x=a=b=c^d^e
ONP: xabcde^^===
INF: x=(a=(b=c^(d^e)))
```

Output

```
ONP: error
INF: a+b+(~a-a)
ONP: ab+a~a-+
ONP: xa~~bc*+=
ONP: ta~x<b~<=
ONP: error
ONP: error
ONP: xa~b*c/d-ef~%+=
INF: x=a+(b+(c+(d+(e+(f+g)))))
INF: a+b+c+d+e+f+g
INF: a+(b+c)+(d+(e+f)+g)
INF: error
ONP: xabc===
INF: x=(a=(b=c))
ONP: xabc^^=
ONP: xabcde^^===
INF: x=(a=(b=c^(d^e)))
ONP: xabcde^^===
```

Submission rules

The .cpp file(s) should be named:

```
Surname_Name_Program_01.cpp
Surname_Name_Program_02.cpp
```

The source files, all other created files, and test text files should be placed in the folder Surname_Name_Laboratory_4.

The files should be submitted to the folder: Programs - laboratory 4 - Tuesday 07:30/09:15.

The folder should be packed as a .rar or .zip archive and submitted to the course folder for Programming Methods - laboratory 4 on delta.pk.edu.pl.